



# From Airflow 2 to Airflow 3: Migrating 100+ DAGs Without Downtime or Developer Burden



Gonalo Costa



# Gonçalo Costa

 SRE, Data Products, Miro

- » Originally from Portugal 🇵🇹  
Now living in the Netherlands 🇳🇱
- » Began career in tech as a Data Engineer,  
focus on Big Data processing. Working with  
Airflow for 9+ years.
- » I work for Miro as an SRE in the Data  
Products domain.
- » First time ever presenting in a conference.

# How the story starts...



Hey! Did you see Airflow 3 came out with some cool new features. So... can we upgrade soon?

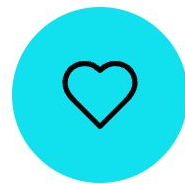


**Pedro**

Data Engineer



# Why should we listen to Pedro



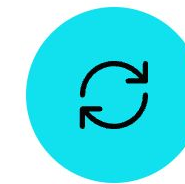
## Version 2 deprecation

We want to be on an actively supported version.



## Cost increases with time

The more we wait the more DAGs and logic is impacted.



## What Airflow 3 unlocks

DAG versioning  
Data assets  
New API server  
Improved Backfill Experience

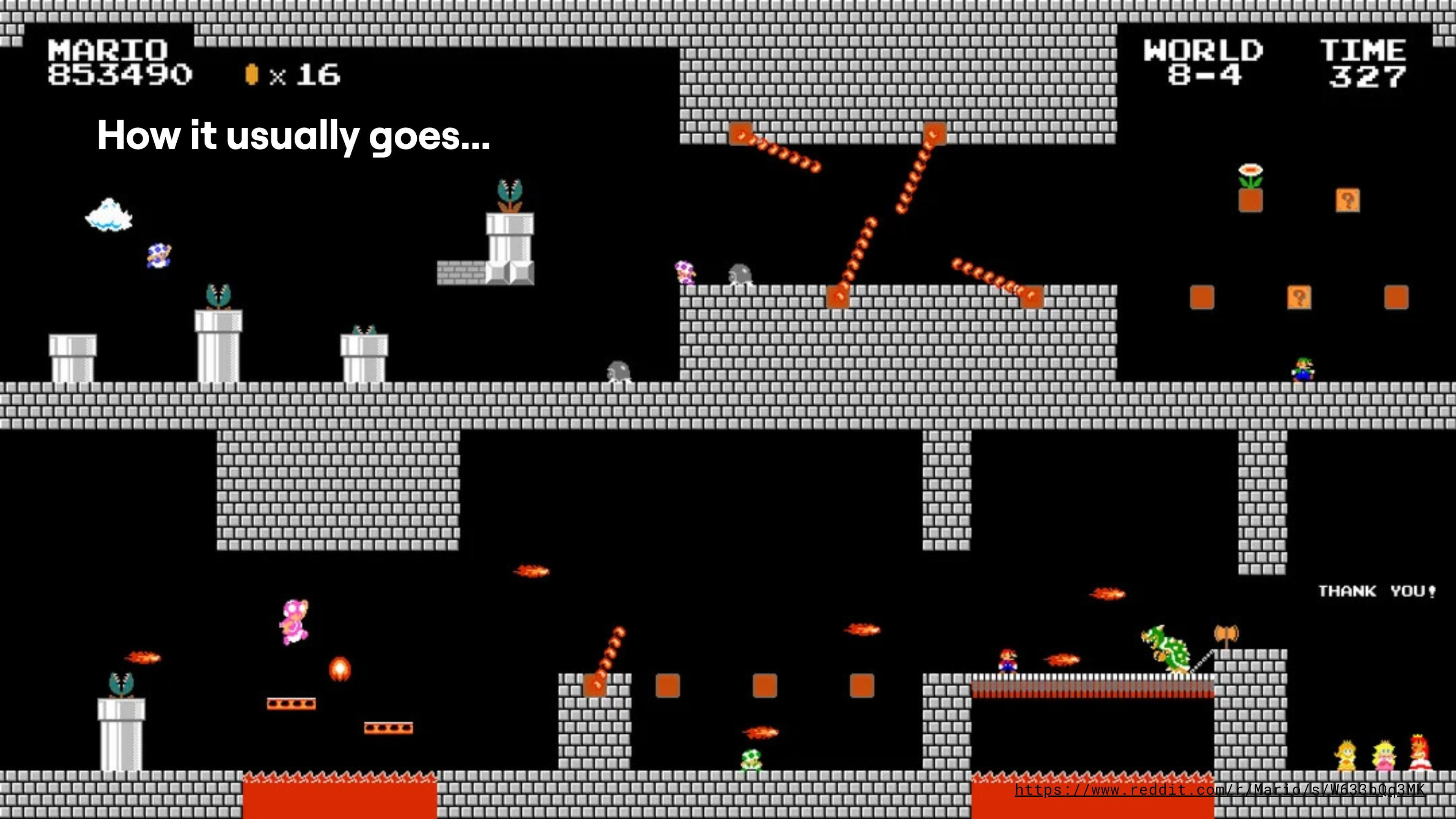
# How do we get there?

MARIO  
853490    🍄 × 16

WORLD  
8-4

TIME  
327

How it usually goes...



# How I would like it to go...



Hey claude, migrate our codebase and infrastructure to Airflow 3. Do not make any mistakes.



**Me**

Site Reliability Engineer



**How can we do this with no  
downtime and 5+ teams involved?**

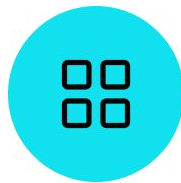
# How we approach these problems

Applying aviation principles to our process

MIRO ENGINEERING · MEDIUM

**From Takeoff to Landing:  
Replacing Cloud  
Infrastructure with Zero  
Downtime**

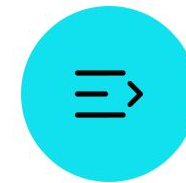
[Read on Medium ↗](#)



## Understand the systems

Go through all the architecture diagrams, and understand which parts are impacted across the systems.

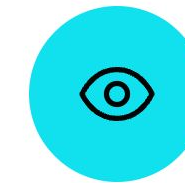
✈️ Study the navigation charts for the multiple stages of a flight.



## Plan the steps

Write a set of checklists with a list of steps that need to be done.

✈️ Pilots use checklists for multiple stages of a flight.

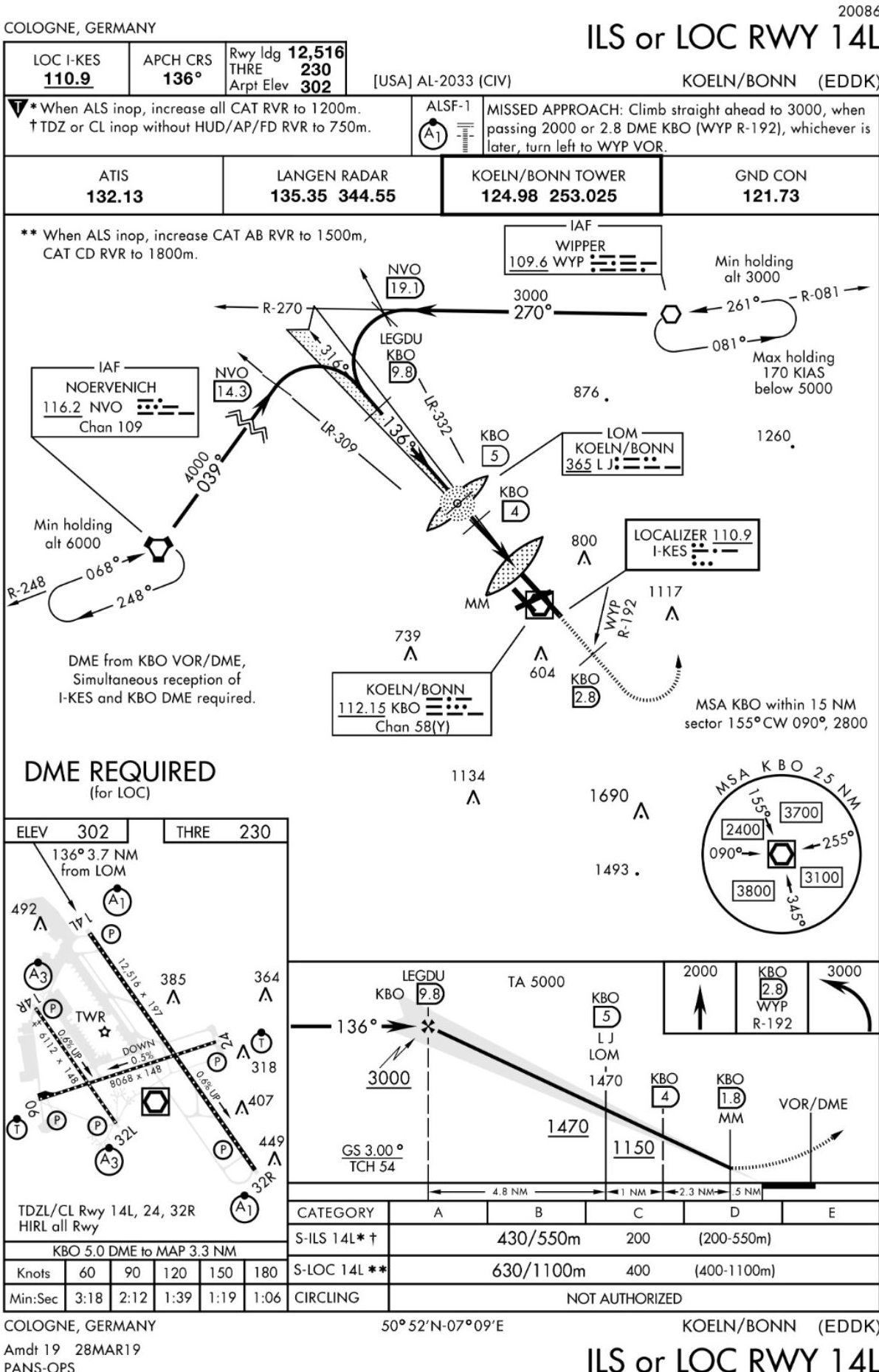


## Make the work visible

Use dashboards to help you visualize the progress, and regularly communicate everything we do.

✈️ Scan the instrument panel to track the multiple systems.

# Understanding



Source: [en.wikipedia.org/wiki/Approach\\_plate](https://en.wikipedia.org/wiki/Approach_plate)

# Our Setup

Airflow 2.10.3 running in three Kubernetes clusters using `KubernetesExecutor` (development, staging and production).

## Everything in a single monorepo:

- 100+ DAG files.
- 1 python library with repeatable logic
- 1 helm chart
- 1 terraform module
- 1 Dockerfile
- 60+ Airflow connections
- 130+ Airflow variables
- 10 Airflow providers

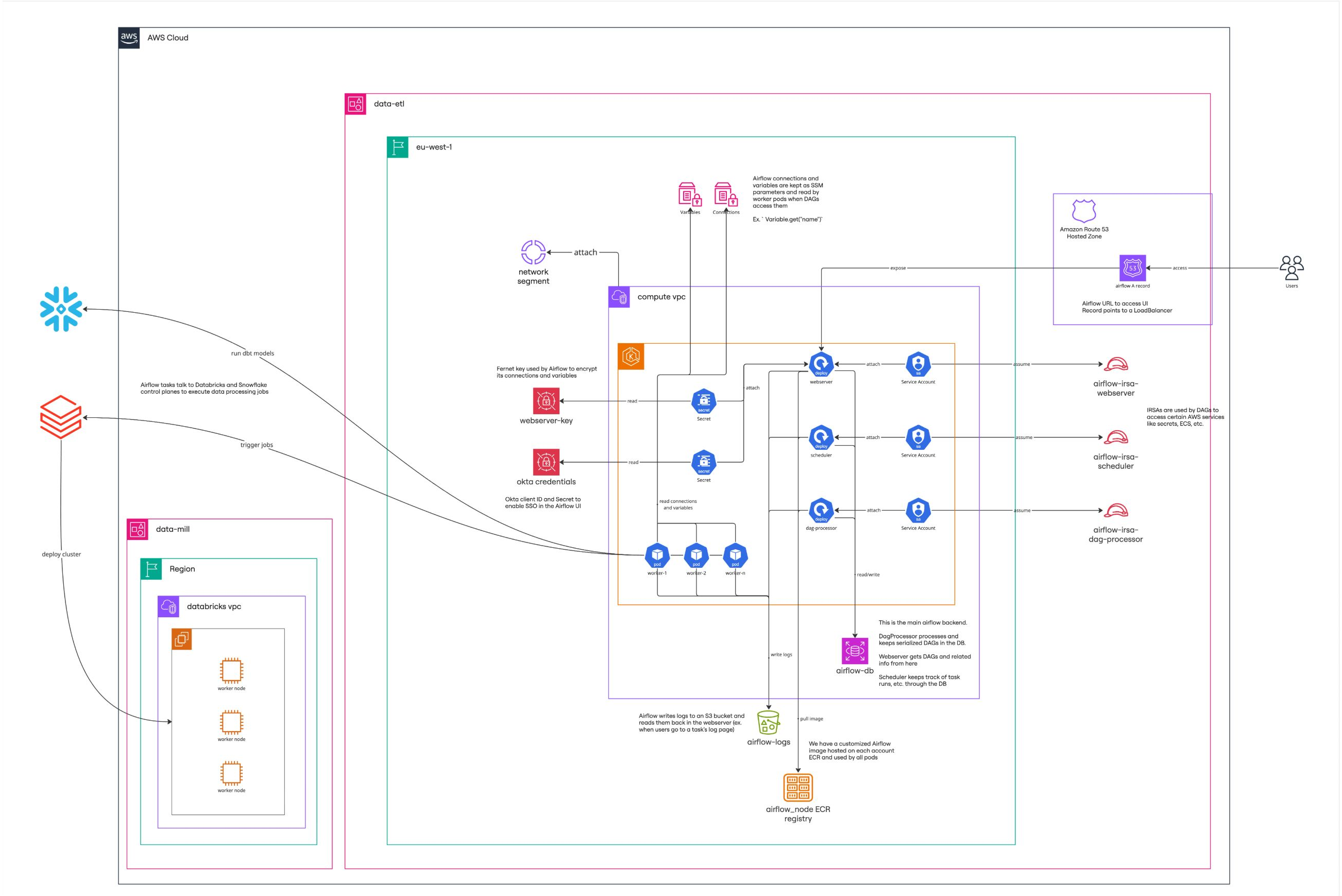
The same DAG file is used by staging and production.

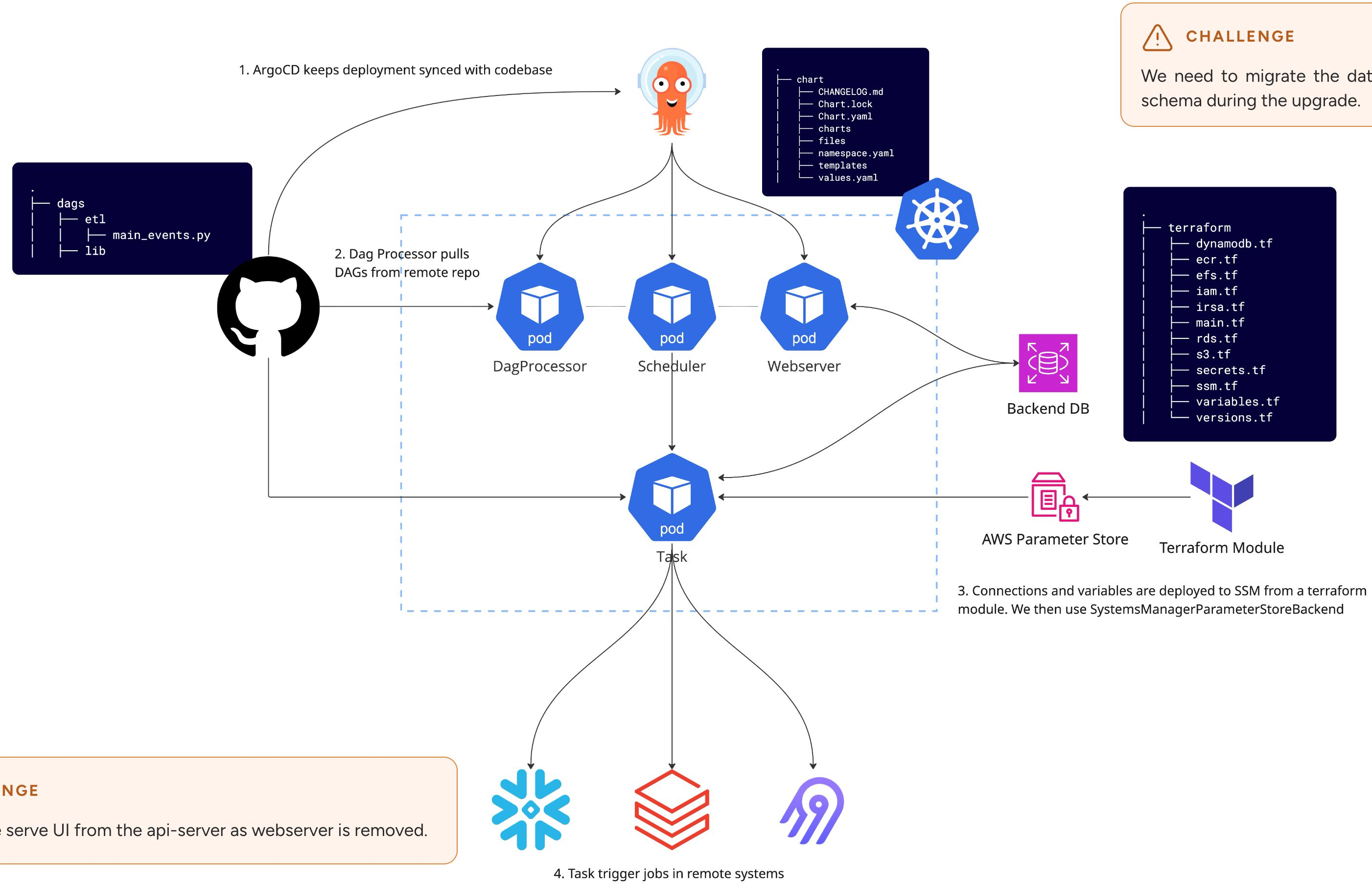


### CHALLENGE

Same DAG code needs to function in both Airflow versions and for different provider versions.

```
.
├── chart
│   ├── CHANGELOG.md
│   ├── Chart.lock
│   ├── Chart.yaml
│   ├── charts
│   ├── files
│   ├── namespace.yaml
│   ├── templates
│   └── values.yaml
├── dags
│   ├── __init__.py
│   ├── etl
│   ├── lib
│   └── production_analytics
├── environments
│   ├── common.yaml
│   ├── component.hcl
│   ├── data-dev
│   ├── data-etl-production
│   └── data-etl-staging
├── Dockerfile
├── pyproject.toml
├── uv.lock
└── terraform
    ├── dynamodb.tf
    ├── ecr.tf
    ├── efs.tf
    ├── iam.tf
    ├── irsa.tf
    ├── main.tf
    ├── rds.tf
    ├── s3.tf
    ├── secrets.tf
    ├── ssm.tf
    ├── variables.tf
    └── versions.tf
```





**CHALLENGE**

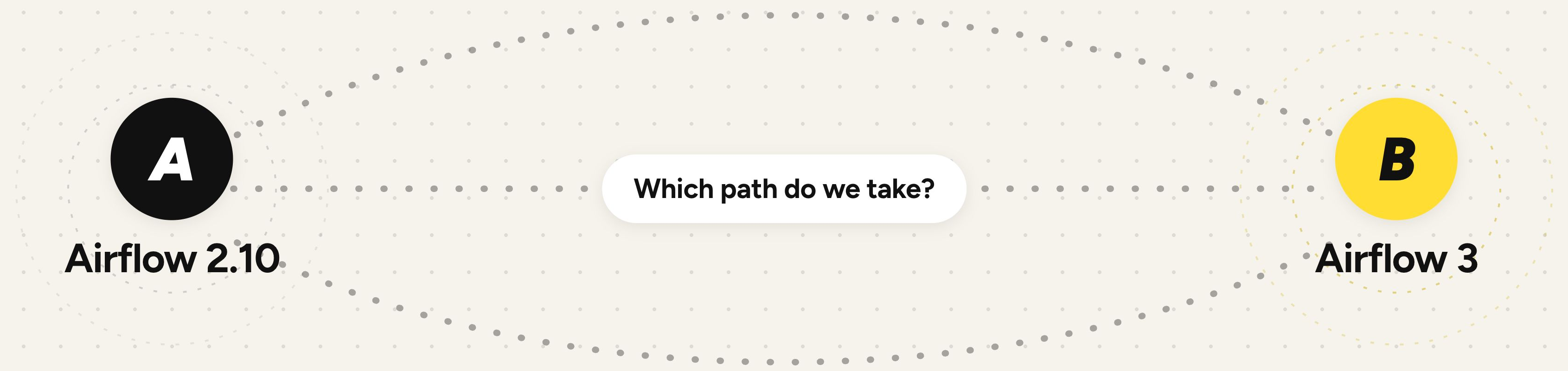
We need to migrate the database schema during the upgrade.

**CHALLENGE**

In Airflow 3 we serve UI from the api-server as webserver is removed.

# What challenges are we facing?

- 1** We need 2 versions of each DAG code: for Airflow 2 and for Airflow 3
  - a. Imports are changing (ex. `airflow.sdk`)
  - b. Some jinja variables were dropped
  - c. Some operators require different arguments
- 2** Airflow providers need to be upgraded
- 3** Airflow connections are changing (ex. schema for snowflake connections)
- 4** ApiServer needs to be enabled in favor of webserver
- 5** We want to make sure as much components as possible are reversible (ex. reversing database after migration job)



# Plan the Steps

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>BEFORE START</b><br>COCKPIT PREP. . . . . COMPLETE (BOTH)<br>GEAR PINS and COVERS. . . . . REMOVED<br>SIGNS. . . . . ON / AUTO<br>ADIRS. . . . . NAV<br>FUEL QUANTITY. . . . . _____ KG.LB<br>TO DATA. . . . . SET<br>BARO REF. . . . . SET (BOTH)<br>WINDOWS/DOORS. . . . . CLOSED (BOTH)<br>BEACON. . . . . ON<br>THR LEVERS. . . . . IDLE<br>PARKING BRAKE. . . . . AS RQRD                                                                                                  | <b>APPROACH</b><br>BRIEFING . . . . . CONFIRMED<br>ECAM STATUS . . . . . CHECKED<br><br>SEAT BELTS . . . . . ON<br>BARO . . . . . SET (BOTH)<br>MDA/DH . . . . . SET (BOTH)<br>ENG MODE SEL . . . . . AS RQRD          |
| <b>AFTER START</b><br>ANTI ICE. . . . . AS RQRD<br>ECAM STATUS . . . . . CHECKED<br>PITCH TRIM . . . . . SET<br>RUDDER TRIM . . . . . ZERO                                                                                                                                                                                                                                                                                                                                         | <b>LANDING</b><br>CABIN CREW . . . . . ADVISED<br>A/THR . . . . . SPEED/OFF<br>ECAM MEMO . . . . . LDG NO BLUE<br>. L/G DOWN<br>. SIGNS ON<br>. CABIN READY<br>. SPLRS ARM<br>. FLAPS SET                              |
| <b>BEFORE TAKEOFF</b><br>FLIGHT CONTROLS . . . . . CHECKED (BOTH)<br>FLT INST . . . . . CHECKED (BOTH)<br>BRIEFING . . . . . CONFIRMED<br>FLAP SETTING . . . . . CONF – (BOTH)<br>V1. VR. V2/FLX TEMP . . . . . _____ (BOTH)<br>ATC . . . . . SET<br>ECAM MEMO . . . . . TO NO BLUE<br>. AUTO BRK MAX<br>. SIGNS ON<br>. CABIN READY<br>. SPLRS ARM<br>. FLAPS TO<br>. TO CONFIG NORM<br>CABIN CREW . . . . . ADVISED<br>ENG MODE SEL . . . . . AS RQRD<br>PACKS . . . . . AS RQRD | <b>AFTER LANDING</b><br>FLAPS . . . . . RETRACTED<br>SPOILERS . . . . . DISARMED<br>APU . . . . . START<br>RADAR . . . . . OFF/STBY                                                                                    |
| <b>AFTER TAKEOFF / CLIMB</b><br>LDG GEAR . . . . . UP<br>FLAPS . . . . . RETRACTED<br>PACKS . . . . . ON<br>BARO REF . . . . . SET (BOTH)                                                                                                                                                                                                                                                                                                                                          | <b>PARKING</b><br>APU BLEED . . . . . ON<br>ENGINES . . . . . OFF<br>SEAT BELTS . . . . . OFF<br>EXT LT . . . . . AS RQRD<br>FUEL PUMPS . . . . . OFF<br>PARK BRK and CHOCKS . . . . . AS RQRD<br>Consider HEAVY RAIN  |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>SECURING THE AIRCRAFT</b><br>ADIRS . . . . . OFF<br>OXYGEN . . . . . OFF<br>APU BLEED . . . . . OFF<br>EMER EXIT LT . . . . . OFF<br>NO SMOKING . . . . . OFF<br>APU AND BAT . . . . . OFF<br>Consider COLD WEATHER |

# Here's what **we will do**...

- PHASE 1**      **Prepare the ground and test**  
While still on Airflow 2. All changes backwards-compatible.  
No impact on running DAGs.
- PHASE 2**      **Flip the infrastructure**  
Bump Helm chart. Staging first, then Production.
- PHASE 3**      **Remove the scaffolding**  
Delete helper code once all envs on AF3.

# Explore and list all required changes

List everything that needs to change.  
Build a checklist of steps.  
Deep dive on challenges we are facing.

## Changeset from 2.10.3 to 3.1.7

By Gonalo Costa 3 min 2 Add a reaction Disabled A A

This page documents every change required to migrate our Airflow deployment and DAG codebase from 2.10.3 to 3.1.7. Changes are grouped by area. Items marked **safe in 2.10.3** can be applied before the infrastructure upgrade without breaking the running environment.

### 1. Infrastructure & Helm

#### 1.1 Helm chart version

Update `Chart.yaml` in each environment to the new chart version (`miro-airflow 8.0.0` wrapping `airflow 1.19.0`).

#### 1.2 Helm values: webserver → api-server

The Airflow webserver has been replaced by a FastAPI-based API server.

```
1 # Before (2.x)
2 webserver:
3   ...
4
5 # After (3.x)
6 apiServer:
7   ...
```

Remove or migrate all `webserver.*` keys to their `apiServer.*` equivalents.

#### 1.3 Remove custom `logging_config_class`

The `logging_config_class` configuration key is no longer supported. Remove it from `values.yaml`. Log configuration is handled via `log_config.py` mounted directly.

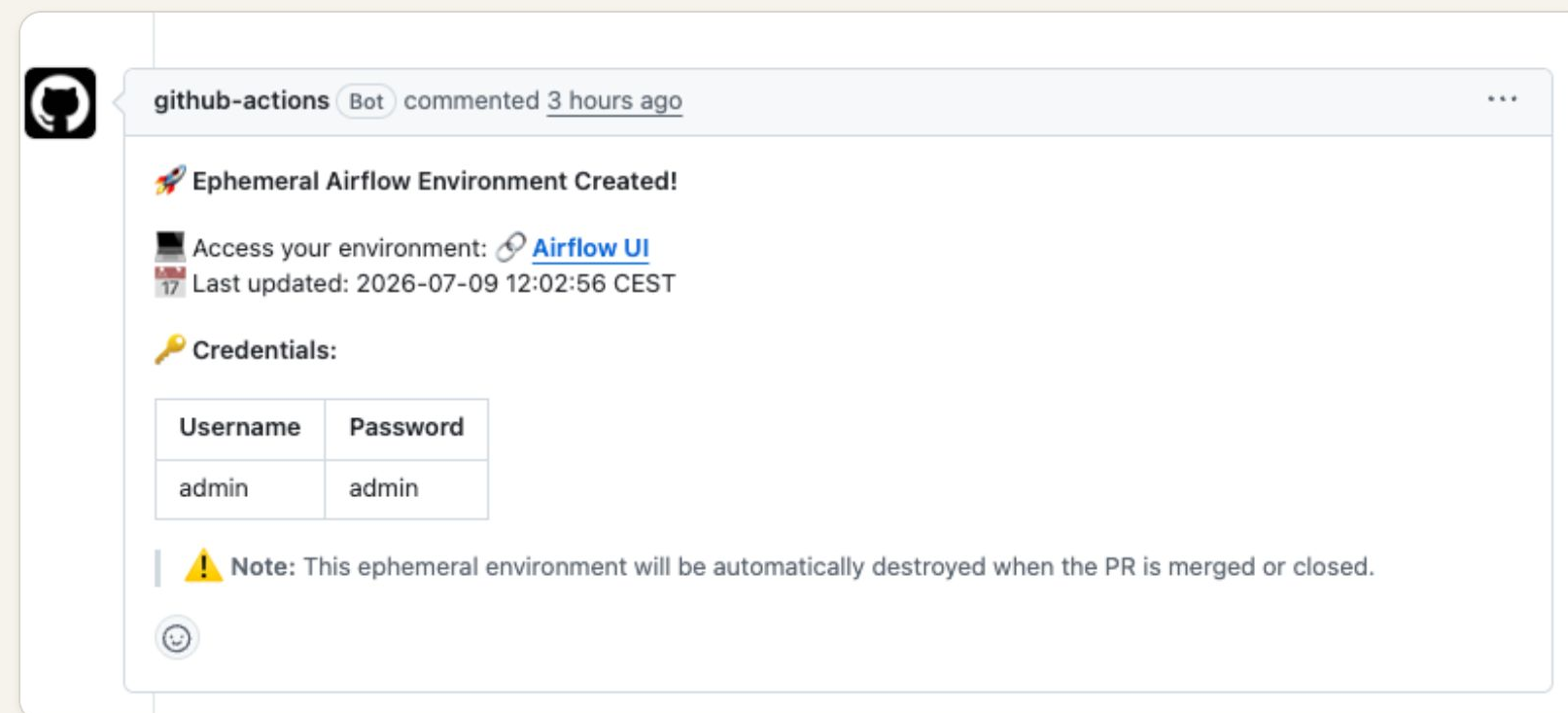
### 2. Docker Image

#### 2.1 Update `AIRFLOW_VERSION` build arg

```
1 ARG AIRFLOW_VERSION=3.1.7
```

# Build and test in development

We deploy a temporary clone of Airflow in its own namespace, triggered by a labeled PR.  
It is used to test changes to DAGs, infra and config in isolation.



## 1 CHALLENGE #1

Run two versions of each DAG code.

## 2 CHALLENGE #2

Attributes are changing for some classes.

## 3 CHALLENGE #3

Airflow connections are changing.

# AI entered the chat...

We gave it the right tools and skills to navigate our infrastructure, troubleshoot and monitor.



## Authenticate locally

AI reads `~/.aws/config` and `~/.kube/config` without any admin profiles configured.

```
kubectl config get-contexts  
→ picks data-ops-dev
```



## Watch pods live

AI uses `kubectl` to watch pod states, stream logs, and surface OOMKilled or CrashLoopBackOff errors in real time.

```
kubectl get pods -n airflow -w  
kubectl logs <pod> -n airflow
```



## Debug failing tasks

When a dry-run failed, AI fetched task logs and drilled into the pod to pinpoint the exact traceback.

```
GET  
/api/v2/dags/{dag}/dagRuns/{run}  
/taskInstances/{task}/logs/1
```

# AI entered the chat...

An agent skill does the heavy lifting. It pulls a failing task's logs and analyses the root cause.

● ● ● airflow-troubleshoot / SKILL.md

---

**name:** airflow-troubleshoot

**description:** Pull a failing task's logs via the REST API and analyse the root cause.

**argument-hint:** <task-url>

**allowed-tools:** curl \*, python3 \*

---

- 1 Parse the task URL → **DAG · run · task** IDs
- 2 Authenticate — **v2 JWT** or **v1 basic auth**
- 3 Fetch task instance + logs via the REST API
- 4 Filter tracebacks → report root cause & fix



# Buiding the compatibility layer

## Python starts

`sitecustomize.py` – runs before any DAG code

## Check airflow version

```
if AIRFLOW_VERSION.split(".")[0] < 3:
```

## We register our custom import Finder

```
sys.meta_path.insert(0, _AirflowSdkCompatFinder())
```

## A DAG imports airflow.sdk

```
from airflow.sdk import Variable
```

## Finder returns new ModuleSpec for `airflow.sdk`

```
find_spec() returns ModuleSpec(name, _AirflowSdkStubLoader())
```

# Buiding the compatibility layer

**1 Create an empty module**  
`create_module() → types.ModuleType("airflow.sdk")`

## Add the Airflow 2 imports

**2** `_AF3_TO_AF2_SYMBOL_MAP` maps  
`airflow.sdk.Variable` to `airflow.models.Variable`  
`setattr(module, sdk_name, getattr(af2_module, af2_attr_name))`

**3 Patch the one changed signature**  
`Variable.get(key, default=...) ≠ default_var=...`

**The DAG gets the correct import under the hood**

# Handling connections

## 01 THE CHALLENGE

Same connections need to work against both the old and the new provider versions during the environment rollout period.

## 02 HOW WE ADDRESSED IT

A custom secrets backend that normalizes connection schemas for different provider versions.

## 03 NORMALIZED ON THE WAY OUT

... af3\_compat\_ssm\_backend.py

```
class AF3CompatSSMBackend(SystemsManagerParameterStoreBackend):
    def deserialize_connection(self, conn_id, value):
        conn = super().deserialize_connection(conn_id, value)
        normalizer = _NORMALIZERS.get(conn.conn_type)
        if normalizer is None:
            return conn  # nothing to fix
        extra = normalizer.normalize(dict(conn.extra_dejson))
        if extra is not None:
            conn.set_extra(json.dumps(extra))
        return conn

class SnowflakeKeyNormalizer(ConnectionNormalizer):
    # provider >=6.3.0 wants base64-encoded PEM
    def normalize(self, extra):
        pk = extra.get("private_key_content") or ""
        if not pk.strip().startswith("-----BEGIN"):
            return None
        pk = base64.b64encode(pk.strip().encode()).decode()
        return {**extra, "private_key_content": pk}
```

### EXAMPLE · SNOWFLAKE

The latest provider now expects base64-encoded private keys.

Our backend encodes the key under the hood based on its version.

# Here's the rough plan

- ✓ Update DAGs to comply with Airflow 3 in a single branch.
- ✓ Deployed new Docker image with Airflow 2 with compat layer baked in.
- ✓ Deploy the ephemeral environment.
- ✓ Setup test DAG and connections to dry-run operators used in prod.
- ✓ Check for import errors and dry-runs results.
- ✓ Upgrade ephemeral environment to Airflow 3.
- ✓ Run test DAGs and dry-run operators again.
- ✓ Deploy compatibility layer to stg and prd.
- ✓ Deploy refactored DAGs and connections.
- ⚠ Switch each environment to new version.

# Making Work Visible



# Seeing changes in real time

Filter resources...

FAVORITES

ServiceAccount

14

Pod

30

ConfigMap

5

Event

6546

Namespace

1

Deployment

5

Ingress

1

WORKLOADS

CronJob

0

DaemonSet

0

Deployment

5

Job

0

Pod

30

ReplicaSet

51

StatefulSet

1

NETWORKING

Ingress

1

NetworkPolicy

0

Service

4

CONFIGURATION

ConfigMap

5

HorizontalPodAutoscal...

0

PodDisruptionBudget

2

Secret

10

STORAGE

39

airflow-statsd-866...

airflow-

| NAME ↓                                 | NAMESPACE ↓ | CONTAINERS                                              | STATUS ↓ | CPU ↓              | MEMORY ↓          | RESTA... ↓ | NODE       |
|----------------------------------------|-------------|---------------------------------------------------------|----------|--------------------|-------------------|------------|------------|
| airflow-api-server-6c8d45df74-6wgm4    | airflow     | <div><div></div><div></div></div>                       | Running  | 21m / 1000m 2%     | 405Mi / 2.0Gi 20% | 0          | ip-10-128- |
| airflow-api-server-6c8d45df74-92mlw    | airflow     | <div><div></div><div></div></div>                       | Running  | 13m / 1000m 1%     | 1.7Gi / 2.0Gi 86% | 0          | ip-10-128- |
| airflow-api-server-6c8d45df74-jbcqc    | airflow     | <div><div></div><div></div></div>                       | Running  | 1004m / 1000m 100% | 543Mi / 2.0Gi 27% | 0          | ip-10-128- |
| airflow-api-server-6c8d45df74-rl78q    | airflow     | <div><div></div><div></div></div>                       | Running  | 11m / 1000m 1%     | 409Mi / 2.0Gi 20% | 0          | ip-10-128- |
| airflow-dag-processor-694f7c7867-q4nmr | airflow     | <div><div></div><div></div><div></div><div></div></div> | Running  | 1854m / 400m 464%  | 757Mi / 5.0Gi 15% | 0          | ip-10-128- |
| airflow-pgbouncer-fcf964696-qqrn4      | airflow     | <div><div></div><div></div></div>                       | Running  | 4m / 250m 2%       | 5Mi / 456Mi 1%    | 0          | ip-10-128- |
| airflow-pgbouncer-fcf964696-sn5z6      | airflow     | <div><div></div><div></div></div>                       | Running  | 5m / 250m 2%       | 4Mi / 456Mi 1%    | 0          | ip-10-128- |
| airflow-pgbouncer-fcf964696-wnjkp      | airflow     | <div><div></div><div></div></div>                       | Running  | 5m / 250m 2%       | 4Mi / 456Mi 1%    | 0          | ip-10-128- |
| airflow-scheduler-fb95885f7-d6f5v      | airflow     | <div><div></div><div></div><div></div></div>            | Running  | 235m / 4500m 5%    | 645Mi / 2.5Gi 25% | 0          | ip-10-128- |
| airflow-scheduler-fb95885f7-flf2l      | airflow     | <div><div></div><div></div><div></div></div>            | Running  | 315m / 4500m 7%    | 834Mi / 2.5Gi 33% | 0          | ip-10-128- |
| airflow-scheduler-fb95885f7-ppnjs      | airflow     | <div><div></div><div></div><div></div></div>            | Running  | 689m / 4500m 15%   | 644Mi / 2.5Gi 25% | 0          | ip-10-128- |
| airflow-statsd-866896d5c6-g9tkj        | airflow     | <div><div></div></div>                                  | Running  | 18m / 250m 7%      | 199Mi / 1.0Gi 19% | 0          | ip-10-128- |
| airflow-triggerer-0                    | airflow     | <div><div></div><div></div><div></div><div></div></div> | Running  | 37m / 1200m 3%     | 593Mi / 2.0Gi 29% | 0          | ip-10-128- |

Pod Running

airflow-statsd-866896d5c6-g9tkj

airflow

Terminal

Logs

Port Forward

YAML

Status

Containers

Environment Variables · 5 vars

Resource Usage

statsd

CPU

0.50 cores

0.25 cores

0.02 cores

(max: 0.03 cores)

Memory

1.86 GiB

954 MiB

199 MiB

(max: 199 MiB)

Last updated: 11:23:14 AM

Conditions (5)

15d 22h

ContainersReady

15d 22h

Ready

15d 22h

PodReadyToStartContainers

15d 22h

Initialized

15d 22h

PodScheduled

App Info

Previous

500 lines

Stream

ERR

WARN

INFO 4

DBG

13:01:16 > INFO Starting StatsD -> Prometheus Exporter main.go:296 {5 fields}

13:01:16 > INFO Build context main.go:297 {5 fields}

13:01:16 > INFO Accepting StatsD Traffic main.go:346 {7 fields}

13:01:16 > INFO Accepting Prometheus Requests main.go:347 {5 fields}



Gonçalo Costa

Mar 9th at 10:22 AM



Upgrade Announcement: Airflow is being upgraded to v3.1.X 

Audience: @data\_products @data-engineering

Action required: Nothing. If this changes I will give a heads up to the required teams.



Point of contact: #tm\_data\_products\_support - Tag me pls @Gonçalo Costa



Summary:

It's been a while since Airflow released a new major version upgrading from 2 to 3. This upgrade introduces a new security model, general improvements, new UI and new features. So far we have been running Airflow 2.10.3 and using already outdated airflow providers for snowflake and databricks. We also want to implement previously requested features like RBAC and build on top of what Airflow is currently recommending and supporting, so it's time for an upgrade.



Impact:

Few breaking changes are introduced by upgrading Airflow and required providers:

1.

Import paths change

2.

Some DAG attributes change (ex. `schedule_interval` is now `schedule`)

3.

Some connections need extra attributes or different attribute types

4.

Some operators require different attributes



Plan:

There are ways to bridge the migration without a "big-bang" strategy. This week we will start working on this plan:

1.

Import paths change - we will implement a shim to temporarily make new imports compatible in Airflow 2 after updating DAG code.

2.

DAG attributes change - the identified changes are backwards compatible with Airflow 2, so we just need to update the code

3.

Operators change - the identified changes are backwards compatible with Airflow 2, so we just need to update the code

4.

Connections attributes change - we will extend Airflow's SSMBackend to make sure current connections are compatible with Airflow 3.

5.

Upgrading staging - After points above are in place and re-tested in an ephemeral environment, staging will be upgraded to AF3

6.

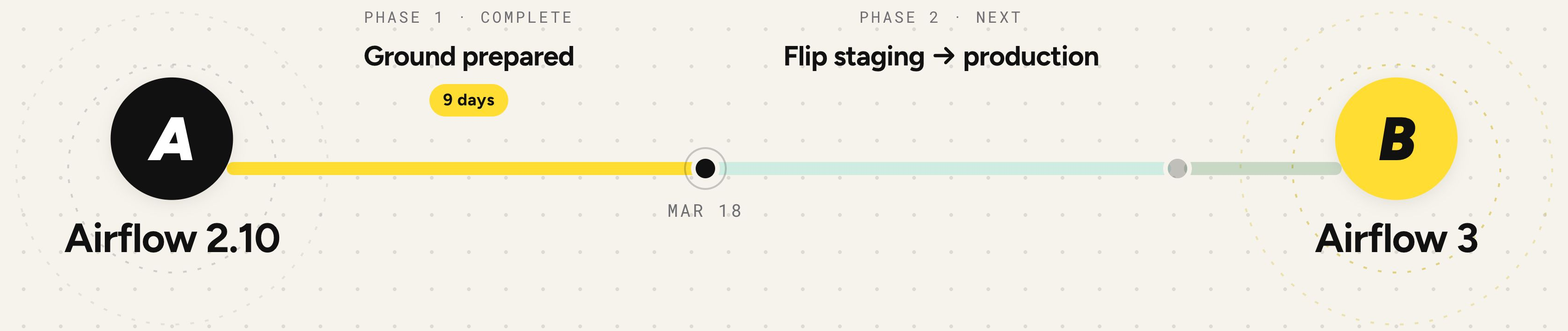
Upgrading production - After everything is tested and we are confident to proceed, production will be upgraded to AF3.

7.

Clean up - Fix connections to use correct attributes (if necessary), then drop SSM custom backend and drop shim.

Communicate the plan

Broadcast the work we are doing. Allow other to subscribe to the updates.





# Switch staging

We picked a time where no DAG was running and upgraded staging:

- 1 Backup backend database
- 2 Upgrade staging Docker image
- 3 Add new image tag in the helm chart `values.yaml`
- 4 Enable database migration job
- 5 If everything went well: disable database migration job in the end.

**We found three issues** 🔥 🔥 🔥

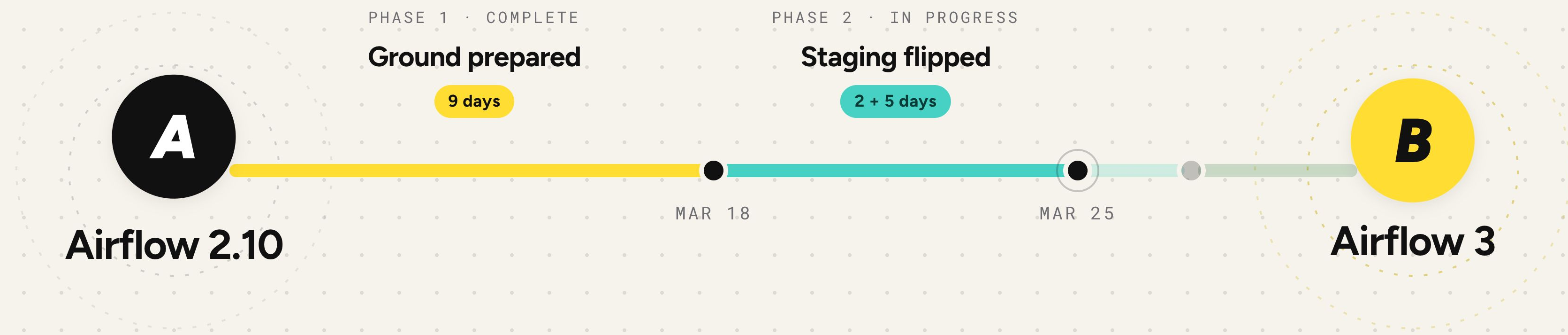
# We found three issues.

|                  |                                                                                                                        |                                                                                                                                     |                                                                                                                                     |
|------------------|------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| NO SCHEDULING    | <p><b>WHAT BROKE</b></p> <p>A DAG scheduled to start running at 11:00 and process data from 10:00 did not trigger.</p> | <p><b>WHY</b></p> <p>Default timetable changed. DAG was firing at 10:00 not 11:00 because of <code>CronTriggerTimetable</code>.</p> | <p><b>FIX</b></p> <p>Use <code>CronDataIntervalTimetable</code> explicitly.</p>                                                     |
| JWT INVALIDATION | <p><b>WHAT BROKE</b></p> <p>In-flight task pods were killed mid-run on every ArgoCD sync.</p>                          | <p><b>WHY</b></p> <p><code>AIRFLOW__API__SECRET_KEY</code> is regenerated on each sync.</p>                                         | <p><b>FIX</b></p> <p>Generate a stable api secret and expose it to the cluster.</p>                                                 |
| HANGING TASKS    | <p><b>WHAT BROKE</b></p> <p>Task pods would spike memory and hang for hours without error logs.</p>                    | <p><b>WHY</b></p> <p>Related to SQL tasks but we couldn't reproduce the issue. Accepted the risk and checked later.</p>             | <p><b>FIX</b></p> <p><u>Temporary</u>: set task timeouts on SQL tasks.<br/><u>Long-term</u>: do python profiling &amp; tracing.</p> |



## TAKEAWAY

We thought we tested everything and still missed 3 problems. Staging was the final validation.



# Switch production

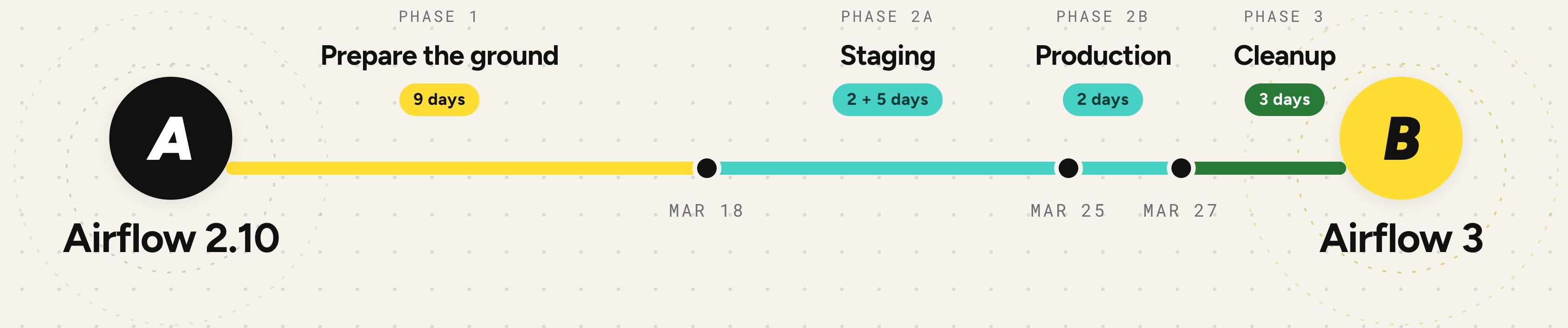
We kept the staging running for a week, then we upgraded production.

**Nothing broken** this time. No downtime. No complaints from users.

# Cleanup helpers

Re-enable standard secrets backend.

Deploy new image without shim and custom secrets backend.



# 21 days start to finish



# Some final **notes**

- 1 Know your systems.** How things work, which changes impact what.
- 2 Version your DAGs.** Make it easier to promote DAGs to different environments and keep track of changes.
- 3 Bootstrap as much as you can before handing over.** Build what you can on your own, then bring the teams in.
- 4 Try to make changes backwards-compatible.** Keep things working on the old version; flip to the new one last.
- 5 Dev doesn't replace staging.** Dev caught ~90% of the problems but staging was the ultimate validation layer.
- 6 Communicate early and often.** Announce up front so everyone is aware of changes and potential problems.
- 7 Use AI, but keep humans in the loop.** Great for bootstrapping code and answering questions, but it also hallucinates a lot.

miro

# Thank you!

Reach out for questions or feedback.



**Connect on LinkedIn**

[/in/goncalo-mcosta](#)



**We're hiring**

[miro.com/careers](https://miro.com/careers)



**Scan to give feedback**